

function `doca_pcc_dev_user_algo()` in `device/rp_main.c`, which dispatches to our handlers in `device/algo/rtt_template.c`.

Why the traffic must use -R. Each RoCE QP negotiates, via ECE, which *algo slot* it uses. Slot 0 is our code; any other slot is the NIC’s built-in default. `perftest`’s `-R (rdma_cm)` is what performs that negotiation — **without -R, the QP silently uses the built-in algorithm and your handlers never fire.**

One call per event, per flow

```
void doca_pcc_dev_user_algo(doca_pcc_dev_algo_ctxt_t *algo_ctxt, // this flow's saved state
                           doca_pcc_dev_event_t      *event,      // opaque event handle
                           const doca_pcc_dev_attr_t  *attr,      // algo_slot, port, ...
                           doca_pcc_dev_results_t     *results)    // OUT: new rate + rtt_req
{
    switch (attr->algo_slot) {
        case 0: rtt_template_algo(event, param, counter, algo_ctxt, results); break; // OUR CODE
        default: doca_pcc_dev_default_internal_algo(algo_ctxt, event, attr, results); // built-in
    }
}
```

The runtime hands you that flow’s **saved context** (`algo_ctxt`) — you read its last rate and write the new one back — and a fresh **results** struct. Filling `results->rate` (and optionally `results->rtt_req`) is the *entire* contract with the hardware. The ACK event is masked off at init (`disable_event_bitmask = DOCA_PCC_DEV_EVNT_ROCE_ACK_MASK`), so the handlers only ever see **new-flow / TX / RTT / CNP / NACK**.

How rate is represented

Rate is a **normalized fixed-point-20 fraction of line rate** — not bits per second:

results->rate	meaning on a 100G port
$1 \ll 20 = 1\,048\,576$	100 % $\approx$ 100 Gb/s
524 288	50 % $\approx$ 50 Gb/s
$\text{MIN_RATE} = 1 \ll 6 = 64$	~ 0.006 % (the floor)

The tunable constants live in `device/algo/rtt_template_algo_params.h`:

```
#define AI ((1 << 20) * 5) / 100 // 52428 = +5% of line rate (additive-increase
↪ unit)
#define NEW_FLOW_RATE (1 << 20) // a brand-new flow starts at full line rate
#define MIN_RATE (1 << (20 - 14)) // 64 = rate floor
#define RATE_MAX (1 << 20) // cap
```

A “ $\times 0.90$ ” cut is expressed in `fxp16` and applied with a fixed-point multiply:

```
#define ECN_CNP_DEC_FACTOR ((1 << 16) * 900) / 1000 // = 58982 →  $\times 0.90$  per CNP
// doca_pcc_dev_fxp_mult(a_fxp16, b_fxp20) == (a*b) >> 16, keeping the fxp20 scale
```

The control loop

`rtt_template_algo()` dispatches by event type. The three handlers that matter:

Decrease — the ECN reaction (CNP). A CNP reaches the sender when the network ECN-marked a data packet (a queue built past the marking threshold) and the receiver echoed it back. Each CNP cuts the rate:

```
static inline void rtt_template_handle_roce_cnp(...) {
    ccctx->flags.was_cnp = 1;
    cur_rate = doca_pcc_dev_fxp_mult(ECN_CNP_DEC_FACTOR, cur_rate); // *0.90
    if (cur_rate < MIN_RATE) cur_rate = MIN_RATE; // floored
    // ... every 500th CNP: doca_pcc_dev_printf("PURE_ECN cnp=%u rate=%u\n", ...)
    results->rate = cur_rate; // the new target rate
    ccctx->cur_rate = cur_rate; // persist for the next event
}
```

So the rate walks down geometrically — $1048576 \rightarrow 943718 \rightarrow 849346 \rightarrow \dots$ — one step per CNP.

Increase — recovery (TX). Every send is a chance to nudge back up; it's gated so recovery is gentle:

```
static uint32_t g_tx_inc = 0;
if ((++g_tx_inc % 1000) == 0) {
    cur_rate += (AI >> 2); // +13107 fxp20  $\square$  +1.25% of line rate
    if (cur_rate > RATE_MAX) cur_rate = RATE_MAX;
}
```

Equilibrium is where the up-nudges balance the CNP down-cuts — the classic DCQCN sawtooth.

RTT — measured, but decoupled. The stock sample steered the rate from RTT here by calling `algorithm_core()`. In the pure-ECN controller that call is **commented out**, so RTT is observed but never changes the rate:

```
// PURE-ECN: rate is driven ONLY by CNP (decrease) and TX (increase).
// To restore the stock RTT/hybrid controller, uncomment the line below.
(void)algorithm_core;
// cur_rate = algorithm_core(ccctx, rtt, cur_rate, param, is_high_tx_util, norm_np_rx_rate);
```

`algorithm_core()` is still in the file (dormant), so switching back to the RTT-based controller is a one-line edit and a rebuild.

How the rate reaches the wire

The surprising part: **your algorithm never programs the shaper directly.**

1. Per event, the DPA runtime calls your handler with a fresh results.
2. You write `results->rate` (an fxp20 fraction of line rate).
3. When your function **returns**, the DOCA PCC infrastructure reads `results->rate` and programs that flow's **per-QP hardware rate limiter** — a token-bucket shaper in the NIC's transmit pipeline (2^{*20} maps to the port's max rate).
4. From then on the NIC **paces that QP's packets** at line speed, with no host or DPA involvement per packet — until the next event invokes your algorithm to revise the set-point.

You write the *set-point*; the NIC does the pacing. One consequence worth remembering for the exercise: the **rate register is not the goodput**. The shaper set-point can sit around ~55 % while aggregate goodput ranges from 44 to 88 Gb/s — because goodput is governed by *queue depth and retransmission waste*, not the average set-point. That's why a *sharper* per-CNP cut (a shallower queue, fewer drops) can *raise* goodput.

Running the exercise (single BlueField-3, loopback)

The card is wired so its two ports form a loopback, and `setup_roce_loopback.sh` puts a sender (`ns1  $\rightarrow$  m1x5_3`) and a receiver (`ns0  $\rightarrow$  m1x5_2`) on it. ECN marking, which a switch would normally do, is done on the card by `doca_flow_ecn_pcap` on **PF0** (`m1x5_0`). The PCC controller attaches to the sender's uplink, **PF1** (`m1x5_1`).

role	device
ECN marker	doca_flow_ecn_pcap on PF0 (mlx5_0)
PCC (RP)	doca_pcc_ecn_rp -d mlx5_1 (PF1)
sender RoCE	mlx5_3 in ns1 (client)
receiver RoCE	mlx5_2 in ns0 (server)

Before you start — the firmware knob

The PCC controller will not start unless the NV-config knob **USER_PROGRAMMABLE_CC=1** is live. Check it with `admin/local_scripts/check_pcc_ready.sh` (it should print ready). Setting it needs a chip reset from the bare-metal host (`mlxfwreset -l 3`) — for the tutorial cards this is already done for you. **REAL_TIME_CLOCK_ENABLE** is **not** required.

Step 1 — Build

Build from the directory that matches your card's DOCA major (doca-2/ for DOCA 2.x, doca-3/ for 3.x). This compiles both the DOCA Flow marker and the PCC controller — the DPA half of the latter is compiled by dpacc at meson-configure time:

```
$ cd doca-2          # or doca-3, whichever matches your card
$ meson setup build && ninja -C build
# => build/doca-flow/doca_flow_ecn_pcap and build/doca-pcc-ecn/doca_pcc_ecn_rp
```

Step 2 — Set up the loopback

```
$ sudo ./admin/local_scripts/setup_roce_loopback.sh
# deletes every OVS bridge and SF on both PFs, then builds ns0/ns1 with mlx5_2 + mlx5_3
```

Step 3 — Start the ECN marker, then traffic (the baseline)

Start the marker on PF0 and leave it running — it CE-marks the wire traffic so the receiver generates CNPs:

```
$ sudo ./build/doca-flow/doca_flow_ecn_pcap -- --pcap /tmp/capture.pcap --percent 100
# the '--' is required: it splits DPDK (EAL) arguments from the app's own arguments.
# watch its per-second line - "CE marked: N" climbing means the eSwitch is marking.
```

Then drive a RoCE flow across the loopback and note the throughput — this is your **baseline**:

```
$ ./run_server.sh      # ib_write_bw receiver in ns0 (10.0.0.1)
$ ./run_client.sh      # ib_write_bw sender in ns1 → 10.0.0.1, prints BW average (~line rate)
```

Step 4 — Load the PCC controller and watch the rate collapse

With traffic still flowing, load the controller on **PF1**. Run it in the **foreground** — it stays Active for the whole window:

```
$ sudo timeout 40 stdbuf -oL ./build/doca-pcc-ecn/doca_pcc_ecn_rp -d mlx5_1 -l 50 > rp.log
↪ 2>&1 &

$ grep PURE_ECN rp.log
PURE_ECN cnp=1    rate=943718
PURE_ECN cnp=501 rate=214380
...
# the rate walking down as CNPs arrive => the loop is live
```

Look back at the `ib_write_bw` numbers: the sender’s throughput **drops** once the controller loads — the reaction point is now cutting its own rate on every CNP. That collapse (baseline → a fraction of it) is the whole point of the exercise.

Stop the controller gracefully when you’re done:

```
$ sudo pkill -INT -x doca_pcc_ecn_rp # never SIGKILL: it leaves a ghost DPA context behind
```

Critical notes — each one silently breaks the demo

1. **-R (rdma_cm) is mandatory on the traffic.** It is what binds the QP to algo slot 0 via ECE; without it your controller never runs. (`run_client.sh/run_server.sh` already pass it.)
2. **Run the controller in the foreground** and drive traffic during its window. Point it at the uplink PF (m1x5_1); the RoCE flow uses the SF (m1x5_3).
3. **Stop only with `pkill -INT`.** A `SIGKILL` (or `docker rm -f`) leaves a ghost DPA context that blocks the next run until a chip reset.
4. **If `grep PURE_ECN` stays empty** while the marker’s CE count climbs, the receiver’s hardware CNP generation may be priority-scoped — add `--tclass=104` (DSCP 26 → TC3) to both `ib_write_bw` ends to steer traffic onto an ECN-enabled priority.

Tuning — the one-line knob

The entire reaction is one constant at the top of `device/algo/rtt_template.c`. Edit it, rebuild (`ninja -C build`), and reload:

```
#define ECN_CNP_DEC_FACTOR (((1 << 16) * 900) / 1000) // ×0.90 per CNP; 800..995 =  
↪ ×0.80..×0.995
```

Measured on a 2→1 incast into a 100G lossless sink:

per-CNP cut	aggregate goodput	CNP	NACK	RTT avg
×0.995	44 Gb/s	124k	130k	56 μs
×0.99	68 Gb/s	63k	67k	31 μs
×0.98	80 Gb/s	31k	34k	—
×0.95	87 Gb/s	12k	15k	32 μs
×0.90	88.5 Gb/s	5.6k	7.8k	17 μs
×0.80	86.7 Gb/s	2.6k	3.8k	6 μs

Takeaway: the shaper set-point sat near ~55 % at *every* setting, yet goodput ranged 44 → 88 Gb/s — because goodput follows queue depth and retransmission waste, not the average rate. A sharper per-CNP cut keeps the queue shallow → fewer retransmissions → higher goodput *and* lower latency. `doca-<major>/doca-pcc-ecn/doca_pcc_ecn_sweep.pdf` plots the full sweep; `tune_ecn.py` reproduces it.

What the controller changed vs. the stock sample

`doca-<major>/doca-pcc-ecn/` derives from NVIDIA’s BSD-3-Clause `rtt_template` PCC sample. The pure-ECN delta is three edits in `device/algo/rtt_template.c`, all revertible through normal git history:

change	why
ECN_CNP_DEC_FACTOR + multiplicative decrease in <code>handle_roce_cnp</code>	the ECN reaction (tuned ×0.90)
gated additive-increase in <code>handle_roce_tx</code>	recovery

change	why
commented-out <code>algorithm_core(...)</code> in <code>handle_roce_rtt</code>	make it ECN-driven, not RTT-driven

To go back to the stock RTT-based controller, uncomment that one `algorithm_core()` line and rebuild — `git log -- doca-2/doca-pcc-ecn/device/algos/rtt_template.c` shows the exact change.